APPLIED AI · PRE-RENEWAL CHECK-IN LOOP

How I built an AI check-in workflow

A pre-renewal resident check-in loop using n8n and OpenAI. Model boundaries, structured outputs, failure paths, real bugs, and the evaluation I'd add next.

One path can send

Five workflows handle outreach, actions, replies, operations and polling. Airtable stores state; Telegram carries manager actions; Gmail carries resident email. Only the action workflow contains a resident send node.

Check-ins, replies and confirmations each need an individual approval.

Outreach → Draft → Manager approval → Gmail

Gmail reply → Match → Classify / route → Manager

The model never picks the resident

OpenAI drafts wording and classifies replies. Code selects cohorts from lease end dates and matches replies by subject token, then exact normalized email. An unmatched reply alerts the manager without assigning anyone. Resident identity never comes from the model's output.

[R-xxxx] → Exact email → Unmatched alert

JSON gives me a contract

The classifier uses a closed JSON Schema: required fields, fixed enums and no additional properties. It returns intent, urgency, grievance details, flags and confidence. Code determines the human-review route. A valid schema constrains the shape of an answer; it doesn't prove the answer is correct.

Schema excerpt · Actual configured enum values

urgency: HIGH | MEDIUM | LOW

confidence: HIGH | MEDIUM | LOW

A valid draft can still fail

After generation, JavaScript checks the assembled check-in: 35-95 words, one question, the resident's first name, and no prohibited terms. Failed drafts get one retry with their violations. A second failure becomes REVIEW_FAILED and reaches the manager without an approve button.

Generate → Validate → Store draft

Failed check → Retry once → REVIEW_FAILED

I checked the draft too early

A manager could edit a draft after its wording check. The text being approved was no longer the text I had validated. I added the same guardrails to the approval path, checking the stored wording used for that send. Human review also needs validation.

Before · Draft → Check → ~~Edit~~ → Send

After · Draft → Check → Edit → Check → Send

Skip could rewind state

Skip reset an already-sent record to PENDING and cleared its draft. A later run could select that resident again. I added a status guard: Skip only accepts pending review states. The fix protects workflow state outside the model call, where another prompt would have changed nothing.

Before · SENT → Skip → ~~PENDING~~

After · SENT → Skip → **Refused**

A failed model call stays visible

When OpenAI returns an error, fixed code emits CLASSIFY_FAILED and routes the reply for human review. Low confidence, grievances, sensitive flags and questions also require a person. Suggested replies remain available, but no resident-facing message leaves without approval. Failure cannot silently become a ~~clean STAYING~~ result.

14/14

isn't classifier ~~accuracy~~

Fourteen check-in drafts passed wording checks in a recorded run. A repeated approval sent nothing; Skip on a sent record was refused; a reply matched by token. The fixture checks verify those behaviors. They do not measure grievance recall or generalization to unseen replies. That's integration

evidence.

Keep the build reproducible

JavaScript builders inline shared libraries into importable n8n workflow JSON. Committed exports contain placeholders; the import helper resolves local identifiers and credentials. Cohort, guardrail and routing logic stay in shared source. Changes can be reviewed in Git instead of surviving only as editor clicks.

Shared libraries → Builders → Workflow JSON → Local import

What I'd change next

I'd build a held-out reply set with labeled grievances, questions and sensitive cases, then measure errors across repeated runs. I'd also add atomic send claims, uncertain-send recovery, paginated reads and backlog processing. Sequential second-tap refusal doesn't prove concurrent approvals or a crash after sending are safe.

Evaluation · Send integrity · Recovery